BACKEND COMPREHENSIVE GUIDE

WhatsApp Clone - Backend Architecture & Interview Q&A Guide

Complete Technical Explanations, Sockets, MySQL Relational Design & System Flows

#01 High-Level Architecture & Tech Stack

Q1 WhatsApp Backend ka overall Architecture aur Tech Stack kya hai?

- **Tech Stack:** Node.js (Runtime), Express.js (REST APIs), Socket.IO (Real-Time Bi-directional WebSockets), MySQL (Relational DB), WebRTC
- **Dual Layer Protocol:** REST APIs use hote hain Initial Fetch, Auth, Profile updates, Search ke liye; jabki Socket.IO use hota hai Messaging, T
- **Connection Pool:** mysql2 connection pool use kiya gaya hai with concurrency limit, auto-reconnection aur prepared statements to prevent SQL
- **In-Memory State:** onlineUsers Map (userId -> Set<socketId>) server RAM me multi-tab & multi-device connections ko O(1) time complexity me

```
onlineUsers = new Map<string, Set<string>>() // Maps userId to active socket IDs
```

[TIP] Interviewer Focus: Interview me batayein ki dual-protocol (REST + WebSockets) network efficiency aur reliability badhata hai.

Q2 Real-Time WhatsApp Ticks (Sent, Delivered, Seen) kaise kaam karte hain?

- **Single Gray Tick (sent):** Jab Sender message bhejta hai, backend use DB me status='sent' save karta hai aur sender ko confirm karta hai.
- **Double Gray Tick (delivered):** Agar Recipient online hai (onlineUsers.has(recipientId)), toh message turant push hota hai aur status='delive
- **Double Blue Tick (seen):** Jab Recipient us chat room ko actively open karta hai ya chat screen par active hota hai, socket event mark_seen
- **Offline Sync:** Jab Recipient offline se online aata hai ('join' event), backend pending messages ko 'delivered' mark karta hai aur sender ko rea

```
Message Status Flow: 'sent' (1 Tick) -> 'delivered' (2 Gray Ticks) -> 'seen' (2 Blue Ticks)
```

[TIP] Interviewer Focus: Har tick transition database me timestamp (delivered_at, seen_at) ke sath persist hoti hai.

#02 Database Schema & Table Relational Design

Q3 Database me kon-kon si tables hain aur unka kya purpose hai?

- **users:** id (Phone number E.164), name, full_phone, about, avatar, avatar_privacy ('everyone'|'contacts'|'nobody'), last_seen.
- **messages:** id, sender_id, recipient_id, text, type ('text'|'image'|'deleted'|'location'), status ('sent'|'delivered'|'seen'), original_text, media_url, creat
- **message_deletions:** message_id (FK), user_id (User who deleted), deleted_at. (Used for 'Delete For Me' per user).
- **blocked_users:** blocker_id, blocked_id, created_at, UNIQUE KEY (blocker_id, blocked_id).
- **contacts:** user_id, contact_id, custom_name, UNIQUE KEY (user_id, contact_id). (Used for private per-user contact aliases/nicknames).

```
CREATE TABLE contacts (user_id VARCHAR(100), contact_id VARCHAR(100), custom_name VARCHAR(150))
```

[TIP] Interviewer Focus: Indexes: conversation_index (sender_id, recipient_id, id) query performance ko 100x optimize karta hai.

#03 Deep Dive: Core Features & Logic

Q4 'Delete For Me' vs 'Delete For Everyone' backend me kaise implement hua?

- **Delete For Me:** Message table se delete nahi hota! Instead, message_deletions table me row insert hoti hai (message_id, user_id).
- **Filtering:** getConversation aur listConversations queries me 'NOT EXISTS (SELECT 1 FROM message_deletions WHERE user_id = :user)'
- **Delete For Everyone:** Sirf sender trigger kar sakta hai. Message table me text='This message was deleted' aur type='deleted' update hota ha
- **Advantage:** Is design se chat backup, audit trail, aur multi-user isolation perfectly maintain rehti hai.

[TIP] Interviewer Focus: Interviewer ko batayein ki Delete For Me me data preserve rehta hai, sirf user ke view se filter out hota hai.

Q5 Message Editing & Edit History backend me kaise track hoti hai?

- **Original Preservation:** Jab message edit hota hai, pehle se stored text ko original_text column me save kiya jata hai.
- **Edited Status:** text column new content se update hota hai aur edited_at column me CURRENT_TIMESTAMP set hota hai.
- **Real-Time Sync:** socket event 'edit_message' sender aur recipient dono ke active clients ko emit hota hai.
- **UI Reflection:** Recipient aur Sender dono ko message ke niche 'Edited' tag dikhta hai aur click karke previous original message view kar sak

```
UPDATE messages SET text = :newText, original_text = COALESCE(original_text, text), edited_at = NOW()
```

[TIP] Interviewer Focus: original_text COALESCE logic ensures ki agar message 2 baar bhi edit ho, toh original pehla message safe rahe.

Q6 User Blocking & Privacy Logic (Profile Photo, Bio, Last Seen) kaise kaam karta hai?

- **Block Table:** blocked_users table me blocker_id aur blocked_id ka unique pair store hota hai.
- **Privacy Masking:** getUser(id, viewerId) function me block status check hota hai. Agar viewer blocked hai, toh avatar=null, about='', lastSeen=n
- **Message Prevention:** addMessage aur socket message event me getBlockStatus check hota hai; blocked hone par message throw error karta
- **Avatar Privacy Modes:** 'everyone' (all users), 'contacts' (only users with previous chat), 'nobody' (no one sees avatar).

[TIP] Interviewer Focus: Block checks dono socket level aur REST controller level par double-guarded hain.

Q7 Private Contact Renaming (Personal Address Book) backend me kaise banaya gaya?

- **Problem:** Agar User 1 User 2 ka naam 'Bhai' save kare, toh User 2 ka profile name change nahi hona chahiye aur doosron ko nahi dikhna chah
- **Solution:** contacts table banayi gayi with composite primary key (user_id, contact_id).
- **Scalar Subquery:** listConversations aur getUser me scalar subquery COALESCE((SELECT custom_name FROM contacts WHERE user_id=:
- **Zero Row Multiplication:** Scalar subquery LIMIT 1 ensure karti hai ki multiple phone variants se duplicate chat rows na banein.

```
PUT /api/chat/contacts/rename -> Body: { userId, contactId, customName }
```

[TIP] Interviewer Focus: Deduplication: Phone digits normalize karke JS layer par strict Set deduplication lagaya gaya hai.

Q8 Swipe-to-Delete Chat (Clear Conversation) backend me kaise kaam karta hai?

- **Batch Deletion:** deleteConversation(userId, otherUserId) dono users ke beech ke saare message IDs ko fetch karke message_deletions table
- **List Filtering:** listConversations query me WHERE m.id IS NOT NULL condition ensure karti hai ki jis conversation ke saare messages dele
- **Resurfacing:** Jab future me doosra user koi naya message bhejta hai, toh naye message ka ID message_deletions me nahi hota, isliye chat a

[TIP] Interviewer Focus: Yeh exact WhatsApp ka native conversation deletion behavior reproduce karta hai.

Q9 Unread Notification Count Badge backend me kaise calculate hota hai?

- **Query Logic:** messages table me WHERE recipient_id = :userId AND status <> 'seen' AND type <> 'deleted' AND NOT IN message_deletions
- **Group By Sender:** Backend total unread count aur per-sender unread count dono return karta hai { totalUnread: 5, bySender: { '+9199...': 3, '+9
- **Real-time Decrement:** Jaise hi user kisi chat par tap karta hai, mark_seen emit hota hai, status 'seen' banta hai, aur unread count turant 0 ho j

```
GET /api/chat/unread/:userId -> Returns totalUnread & breakdown by senderId
```

[TIP] Interviewer Focus: Green badge WhatsApp me total unread count tab bar me aur per-chat unread pills chat row me render karta hai.

Q10 WebRTC Voice & Video Calling me Backend / Socket.IO ka kya role hota hai?

- **Signaling Server:** WebSockets P2P media audio/video stream handle nahi karta, balki Signaling Metadata (Handshake) exchange karta hai.
- **SDP Offer & Answer:** Caller SDP Offer create karta hai -> Backend Recipient ko bhejta hai ('incoming_call') -> Recipient accept karke SDP An
- **ICE Candidates:** Network routing ke liye ICE Candidates socket ke through exchange hote hain ('ice_candidate').
- **P2P Direct Stream:** Handshake complete hone ke baad audio/video direct browser-to-browser WebRTC PeerConnection ke through flow hota

[TIP] Interviewer Focus: Backend is only a signaling gateway; actual media bandwidth server par load nahi daalta.

#04 Complete REST API Reference

API METHOD	ENDPOINT URL	PURPOSE & DESCRIPTION
POST	`/api/auth/send-otp`	Send 6-digit OTP to mobile number
POST	`/api/auth/verify-otp`	Verify OTP & login/register user
GET	`/api/chat/conversations/:userId`	Get active chat list with unread counts
DELETE	`/api/chat/conversations/:otherUserId`	Soft delete / clear chat for current user
GET	`/api/chat/history`	Get past chat history between 2 users
GET	`/api/chat/profile/:id`	Get profile details evaluated for viewer
PUT	`/api/chat/profile/:id`	Update Name, Bio, Avatar, Privacy mode
PUT	`/api/chat/contacts/rename`	Save private per-user contact alias/nickname
GET	`/api/chat/users/search`	Search registered users in DB by number
POST	`/api/chat/block`	Block user from messaging and calling
POST	`/api/chat/unblock`	Unblock contact
GET	`/api/chat/unread/:userId`	Get total and per-sender unread message counts

#05 Complete Socket.IO Events Reference

EVENT NAME	DIRECTION	PAYLOAD & ACTION
join	`Client -> Server`	{ userId } : Register socket room & go online
message	`Client -> Server`	{ to, text, type, mediaUrl } : Send message
message_saved	`Server -> Sender`	{ message } : Outgoing message saved with status
message_received	`Server -> Recipient`	{ message } : Push new incoming message in real-time
mark_seen	`Client -> Server`	{ otherUserId } : Turn messages to Double Blue Ticks
typing	`Client -> Server`	{ to, isTyping } : Broadcast typing indicator
delete_message	`Client -> Server`	{ messageId, deleteFor: 'me'|'everyone', to }
edit_message	`Client -> Server`	{ messageId, text, to } : Edit message text
delete_conversation	`Client -> Server`	{ otherUserId } : Clear conversation
rename_contact	`Client -> Server`	{ contactId, customName } : Private alias sync
call_user	`Client -> Server`	{ to, callerName, offer, callType: 'voice'|'video' }
call_accepted	`Client -> Server`	{ to, answer } : WebRTC SDP Answer exchange
ice_candidate	`Both Ways`	{ to, candidate } : WebRTC ICE candidate sync
disconnect / logout	`Client -> Server`	Update last_seen and broadcast offline status